

INITIAL FINDINGS

Hybrid Transactional and AI-Native Database

A unified platform for relational, document, full-text and vector workloads

Prepared for Abhigith M Nair

Version 1.0 | 13 June 2026

Working product statement

A transactional database platform for application and AI data, combining SQL, JSON, full-text search, vector search, permissions and RAG workflows in one developer-friendly system.

Document purpose

This document consolidates the complete initial discussion and research for a database that can handle both conventional application workloads and AI workloads. It covers the product concept, proposed capabilities, architecture, query model, implementation approach, market landscape, differentiation opportunities, risks and recommended strategy.

Status and scope

- Status: Initial findings and feasibility assessment; not yet a final product requirements document.
- Scope: Single-node database concepts, AI/vector functionality, market comparison and an MVP delivery path.
- Primary audience: Founder/developer, technical collaborators, potential advisers and early evaluators.
- Assumption: The initial implementation team is strongest in C#, .NET, React, SQL Server and Azure.

Contents

1. Executive summary
2. Problem and product vision
3. Core capabilities
4. Query and data model
5. AI and RAG workflows
6. Proposed architecture
7. Storage and transaction design
8. Vector search design
9. Implementation plan
10. Market landscape
11. Differentiation strategy
12. Strategic options and recommendation
13. Risks and success criteria
14. Roadmap and next actions
15. References

1. Executive summary

The proposed product is a hybrid database that stores and queries normal application data and AI-oriented data in one system. It would combine relational tables, transactions and business constraints with JSON documents, full-text search, vector embeddings, semantic similarity search and RAG-oriented workflows.

Key finding

The market already contains strong databases that combine operational data and vector search. Therefore, “SQL plus vectors” is a valid foundation but is not, by itself, a defensible product advantage.

The strongest opportunity is to make the complete AI data lifecycle easier and safer: automatic document ingestion and chunking, embedding lifecycle management, secure retrieval, consistency between source data and search indexes, observability, model/version tracking and a first-class .NET/Azure developer experience.

Recommended direction

- For deep learning and database-engine research: build a small single-node engine from scratch, beginning with a durable key-value store and later adding tables, SQL and exact vector search.
- For a commercially realistic MVP: build an AI-native data platform on PostgreSQL and pgvector instead of rebuilding storage, transactions, backup, replication and SQL correctness.
- Position the product as a secure, developer-first AI data platform rather than merely another vector-enabled SQL database.
- Use C#/.NET for the control plane, ingestion workers, SDK and APIs; React for the management studio; PostgreSQL/pgvector for the durable data plane.

2. Problem and product vision

2.1 Current application architecture problem

Modern AI applications frequently require multiple systems: a relational database for users, permissions and transactions; object storage for documents; a search engine for keywords; and a vector database for embeddings. Developers then have to synchronize records, permissions, deletions, model versions and indexes across those systems.

Typical current architecture

Application

- |-- SQL database: users, orders, permissions, transactions
- |-- Object storage: original files
- |-- Search engine: exact keyword and full-text retrieval
- |-- Vector database: embeddings and semantic retrieval
- `-- Custom pipeline: chunking, synchronization and retries

The proposed database/platform reduces this fragmentation by creating a single logical data system in which structured data, flexible metadata, text and vector embeddings can be queried together.

2.2 Product vision

Vision

One database experience for operational applications and AI applications, where relational filters, security rules, keyword search and semantic search can be executed together.

2.3 What “better than SQL” should mean

SQL is a query language, not a database category to be replaced. A better framing is to retain relational strengths while reducing the operational complexity of modern AI data. The target advantages are:

- Relational correctness: transactions, constraints, joins and security.
- Document flexibility: JSON metadata and schema evolution.
- AI retrieval: native vector types, similarity functions and indexes.
- Search quality: full-text plus semantic hybrid ranking.
- AI lifecycle automation: ingestion, chunking, embeddings, re-indexing and model versioning.
- Developer simplicity: one SDK, one policy model and one observable query path.

3. Core capabilities

3.1 Relational and operational capabilities

- Tables and typed columns
- Primary and foreign keys
- Unique, required and check constraints
- Transactions and isolation
- Joins, filtering, sorting and aggregation
- Conventional B+ tree and hash indexes
- Row-level and tenant-level security
- Backup, restore and audit history

3.2 Flexible document capabilities

- JSON columns for variable metadata
- JSON path queries
- Optional JSON path indexes
- Safe schema evolution and schema versions
- Structured and unstructured attributes in the same record

3.3 AI and search capabilities

- Dense vector columns with dimension validation

- Cosine, Euclidean and dot-product similarity
- Exact nearest-neighbour search
- Approximate nearest-neighbour indexes such as HNSW and IVF
- Full-text inverted indexes
- Hybrid lexical and semantic ranking
- Reranking support
- Embedding model and content-version metadata

3.4 Platform capabilities

- Automatic document chunking and overlap policies
- Embedding job orchestration with retries
- Detection and rebuilding of stale embeddings
- Secure RAG queries with permission filtering
- Retrieval history, scores, latency and user-feedback tracking
- Auto-generated REST/GraphQL endpoints where appropriate
- .NET SDK, command-line tools and React management studio

4. Query and data model

4.1 Unified table example

```
CREATE TABLE products (
  id UUID PRIMARY KEY,
  name TEXT REQUIRED,
  category TEXT,
  price DECIMAL,
  description TEXT,
  metadata JSON,
  embedding VECTOR(1536),
  created_at TIMESTAMP DEFAULT NOW()
);
```

4.2 Normal relational query

```
SELECT *
FROM products
WHERE category = 'Laptop'
AND price < 80000;
```

4.3 Semantic query

```
SELECT *
FROM products
ORDER BY COSINE_DISTANCE(embedding, @queryVector)
LIMIT 10;
```

4.4 Hybrid query

```
SELECT *
FROM products
WHERE category = 'Laptop'
AND price < 80000
ORDER BY COSINE_DISTANCE(embedding, @queryVector)
LIMIT 10;
```

Hybrid querying is the central value: semantic similarity is combined with deterministic business filters such as tenant, price, status, dates, ownership and permissions.

4.5 Historical and temporal query concept

```
SELECT *
FROM customers
AS OF '2026-06-01 10:00:00';

SELECT *
FROM employees
HISTORY
WHERE id = 100;
```

4.6 Suggested initial types

Category	Types	Notes
Scalar	INT, LONG, DOUBLE, DECIMAL, BOOLEAN	Normal application values
Identity/time	UUID, DATETIME, TIMESTAMP	Keys and temporal data
Text/flexible	TEXT, JSON	Content and metadata
AI	VECTOR(n)	Fixed-dimension embeddings

5. AI and RAG workflows

5.1 Recommended document model

```
CREATE TABLE documents (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  title TEXT,
  source_url TEXT,
  metadata JSON,
  created_at TIMESTAMP
);

CREATE TABLE document_chunks (
  id UUID PRIMARY KEY,
  document_id UUID REFERENCES documents(id),
  chunk_number INT,
  content TEXT,
  token_count INT,
  embedding VECTOR(1536)
);
```

Documents should normally be divided into chunks because retrieval needs focused passages rather than one large vector for an entire file.

5.2 Permission-aware retrieval

```
SELECT chunk.content
FROM document_chunks chunk
JOIN documents doc ON doc.id = chunk.document_id
JOIN user_permissions permission
  ON permission.document_id = doc.id
WHERE permission.user_id = @userId
ORDER BY COSINE_DISTANCE(chunk.embedding, @queryVector)
LIMIT 5;
```

5.3 Automatic AI index concept

```
CREATE AI INDEX document_search
ON documents(content)
WITH (
  chunk_size = 500,
  overlap = 50,
  embedding_model = 'configured-embedding-model'
);
```

The platform would detect inserted or changed source content, split it, create embeddings, store provenance, update indexes and remove stale chunks.

5.4 Embedding lifecycle information

- Source record and content hash
- Content version
- Chunking strategy and chunk version
- Embedding provider and model
- Model version and dimensions
- Generation time and job status

- Index version
- Stale/current/error state

5.5 Secure RAG concept

```
RAG SEARCH policies
FOR 'What are the maternity leave rules?'
AS USER @userId
WHERE tenant_id = @tenantId
LIMIT 5;
```

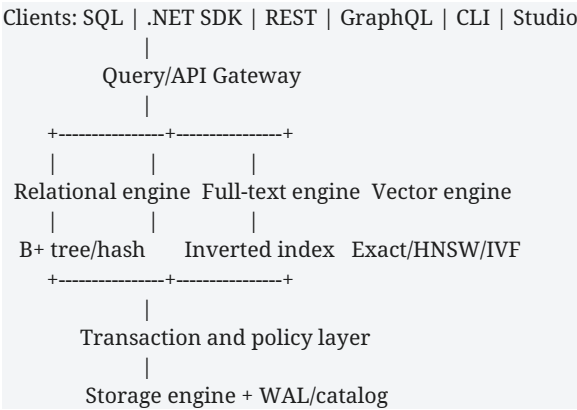
This higher-level syntax is illustrative. Internally, it should enforce tenant isolation, row-level permissions, classification policies and audit logging before content is made available to a model.

5.6 AI observability

- Question and normalized search query
- Filters and security context
- Candidate and selected chunks
- Vector, keyword and reranking scores
- Model/provider versions
- Latency and token usage
- Generated answer reference
- User feedback and failure classification

6. Proposed architecture

6.1 Logical architecture



6.2 AI platform architecture recommended for MVP

Application / .NET SDK / REST

- |
- AI-aware service layer
- collections and schemas
- ingestion and chunking
- embedding workers
- secure retrieval
- observability

- |
- PostgreSQL + pgvector
- ACID data and SQL
- JSON and full-text
- vector columns/indexes
- backup and replication

|

React management studio

6.3 Proposed technology stack

Area	Recommended technology	Reason
Core services	C# / modern .NET	Matches current expertise and offers strong async, memory and server tooling
Durable data plane	PostgreSQL + pgvector	Mature SQL, ACID, recovery, JSON, full-text and vectors
API	ASP.NET Core; gRPC internally where useful	Strong .NET ecosystem and streaming support
Client	First-class .NET SDK	Primary differentiating developer experience
Studio	React	Existing skill set and broad UI ecosystem
Workers	Background .NET services	Chunking, embedding and re-indexing jobs
Deployment	Docker; Azure-friendly hosting	Repeatable local and cloud operation

7. Storage and transaction design - from-scratch research route

This section describes how to build an educational or research database engine from scratch. It is not the recommended commercial MVP path.

7.1 Layered development order

1. Durable append-only key-value engine

2. Tables, schemas and typed rows
3. Minimal SQL parser and execution engine
4. Primary and secondary indexes
5. Vector type and exact similarity search
6. Transactions, write-ahead log and recovery
7. Approximate vector indexes
8. Server protocol, SDK and management studio
9. Replication and distributed operation only after single-node correctness

7.2 Initial repository structure

```
FusionDb/
src/
  FusionDb.Core/
  FusionDb.Storage/
  FusionDb.Schema/
  FusionDb.Query/
  FusionDb.Transactions/
  FusionDb.Vector/
  FusionDb.Server/
  FusionDb.Client/
tests/
  FusionDb.Storage.Tests/
  FusionDb.Query.Tests/
  FusionDb.Vector.Tests/
  FusionDb.Recovery.Tests/
studio/
  fusiondb-studio/
```

7.3 Append-only key-value foundation

```
Record layout
+-----+-----+-----+-----+
| Record type | Key length | Value length | Checksum |
+-----+-----+-----+-----+
| Key bytes           |
+-----+
| Value bytes         |
+-----+
```

Each update is appended rather than overwriting an existing record. An in-memory index maps keys to current file offsets. Startup recovery scans valid records and rebuilds the index. Compaction rewrites only current records into a clean file.

```
public interface IStorageEngine : IDisposable
{
    ValueTask PutAsync(ReadOnlyMemory<byte> key,
        ReadOnlyMemory<byte> value,
        CancellationToken cancellationToken = default);

    ValueTask<byte[]?> GetAsync(ReadOnlyMemory<byte> key,
        CancellationToken cancellationToken = default);

    ValueTask DeleteAsync(ReadOnlyMemory<byte> key,
        CancellationToken cancellationToken = default);
}
```

7.4 Tables and binary rows

- Catalog storing table and column metadata
- Schema version embedded in each row
- Null bitmap
- Fixed-length values stored inline
- Offsets for variable-length text, JSON and vectors
- Internal key convention using table identifier and primary key

7.5 SQL parser and execution

The first grammar should support CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, WHERE, ORDER BY and LIMIT. It should deliberately exclude complex joins, window functions, procedures, triggers and recursive CTEs.

```
SELECT name
FROM users
WHERE age > 25
ORDER BY name
LIMIT 10;
```

Execution tree:

```
Projection(name)
  Sort(name)
    Filter(age > 25)
      TableScan(users)
```

7.6 Transactions and recovery

```
BEGIN TX-100
UPDATE account-1 old=5000 new=4000
UPDATE account-2 old=1000 new=2000
COMMIT TX-100
```

Write-ahead logging records changes before data pages are considered durable. Startup recovery replays committed operations and ignores or rolls back incomplete transactions. Basic shared/exclusive locking should precede a later MVCC implementation.

8. Vector search design

8.1 Vector representation

The initial vector type can be stored as 32-bit floating-point values. A 384-dimensional vector requires about 1,536 bytes before row/index overhead, while a 1,536-dimensional vector requires about 6,144 bytes.

8.2 Similarity functions

- Cosine similarity/distance for angle-based semantic similarity
- Euclidean (L2) distance
- Inner product/dot product
- Optional L1, Hamming or Jaccard for specialized vector types later

8.3 Exact search first

The MVP should calculate the distance from the query vector to each eligible row, retain the best K candidates in a bounded priority queue and return sorted results. This is easy to verify and suitable for small datasets.

8.4 Approximate search later

Index	Strength	Trade-off	Recommended timing
HNSW	Strong speed/recall and no training stage	High memory and more complex persistence/updates	After exact search and transactions
IVF/IVFFlat	Scales by searching selected clusters	Needs training/tuning and can reduce recall	Later for larger datasets
Exact scan	Perfect recall and simple correctness	Linear cost with record count	MVP and validation baseline

8.5 Hybrid execution planning

The planner must decide whether to filter structured data first or search vector candidates first. For the first implementation, apply security and relational filters first, then calculate vector scores. Later, the planner can estimate selectivity and choose a more efficient plan without violating access rules.

9. Implementation plan

9.1 MVP scope

- PostgreSQL-based collections/tables
- JSON metadata
- Vector columns using pgvector
- Exact and indexed vector retrieval
- Full-text plus vector hybrid search
- Tenant and user permission filters
- Document ingestion and chunking
- External embedding-provider integration
- Embedding status/version tracking
- .NET SDK and REST API
- React administration and retrieval-debugging UI

9.2 Deliberately excluded from first release

- New disk/storage engine
- Custom SQL optimizer
- Distributed consensus and sharding

- General-purpose replacement for PostgreSQL or SQL Server
- In-database execution of arbitrary LLMs
- Support for every embedding provider and data modality
- Complex autonomous agents

9.3 Suggested service modules

HybridDb.Api	- public REST/control endpoints
HybridDb.Core	- domain models and policies
HybridDb.Postgres	- schema, SQL and pgvector access
HybridDb.Ingestion	- documents, parsers and chunking
HybridDb.Embeddings	- provider abstraction and workers
HybridDb.Search	- lexical/vector fusion and reranking
HybridDb.Security	- tenant and document authorization
HybridDb.Observability	- traces, scores, audit and feedback
HybridDb.Client	- .NET SDK
HybridDb.Studio	- React administration UI

9.4 First 30-day prototype plan

Week	Focus	Demonstrable outcome
1	PostgreSQL/pgvector setup, schema, .NET persistence and vector insert/query	Store business records and embeddings together
2	Document ingestion, chunking, provider abstraction and job status	Upload text/document content and generate versioned chunks
3	Hybrid search, metadata filters, tenant security and retrieval API	Permission-aware semantic search with structured filters
4	.NET SDK, React retrieval inspector, logs, feedback and demo application	End-to-end secure RAG retrieval demonstration

10. Market landscape

The market assessment was checked against official product documentation available in June 2026. Capabilities and preview status can change; the cited product documentation should remain the source of truth for implementation decisions.

Product	Operational model	AI/vector position	Implication
PostgreSQL + pgvector	Relational SQL, ACID, JSON, joins and mature ecosystem	Exact and approximate nearest-neighbour search; HNSW and IVFFlat	Most practical open-source foundation and direct competitor
SQL Server 2025 / Azure SQL	Enterprise relational platform closely aligned	Native vector data type/functions;	Strong enterprise alternative for

	with .NET/Azure	approximate vector index/search documented as preview in current docs	Microsoft customers
Oracle AI Database	Enterprise relational and JSON database	Vector type, indexes, semantic search and combination with relational business data	Powerful enterprise benchmark
MongoDB Atlas	Document operational database	ANN/ENN vector search, full-text combination and metadata pre-filtering	Strong document-first competitor
SingleStore	Distributed SQL and analytics	Native vectors, kNN/ANN, vector indexes and hybrid full-text/vector search	Close unified performance-oriented competitor
Elasticsearch	Search-oriented document engine	Dense/sparse vectors, filters, aggregations, hybrid retrieval and RRF	Strong search-first competitor, less relational/transaction centric
MySQL HeatWave	MySQL plus HeatWave cloud services	In-database vector store, ingestion, RAG routines and vector index automation	Shows movement toward end-to-end AI data pipelines

10.1 Market conclusion

Conclusion

The concept is established and commercially important, but a new product cannot differentiate only by adding a VECTOR column and similarity function. Mature vendors already provide these capabilities.

11. Differentiation strategy

11.1 Automatic AI indexing

Provide a declarative AI index that owns content detection, chunking, embedding generation, retries, indexing, deletion and refresh. The developer should not have to build a separate synchronization service for every application.

11.2 Embedding lifecycle management

- Detect stale embeddings when source content changes
- Track embedding model/provider/version and dimensions
- Support controlled background migration to a new model
- Show partial-failure and retry status
- Preserve old index until the new index is validated
- Measure recall/quality changes across versions

11.3 Secure RAG as a database capability

- Authorization before retrieval
- Tenant boundary enforcement
- Document-level and row-level policies
- Classification and sensitivity filters
- Complete audit trail of retrieved context
- Prevention of unauthorized candidate leakage

11.4 Transactional consistency for AI search

Define clear consistency states for source rows, chunks and embeddings. A commit should either make a record searchable under a defined policy or expose a visible pending/error status. The system must not silently return stale or orphaned context.

11.5 AI observability and evaluation

- Retrieval inspector showing why each result appeared
- Keyword/vector/reranker score decomposition
- Bad-query and missing-content analysis
- User feedback linked to retrieved context
- Latency, cost and token measurements
- Offline benchmark sets and regression testing

11.6 First-class .NET and Azure experience

```
var collection = database.AICollection<PolicyDocument>(
    x => x.Content,
    new AiIndexOptions
    {
        ChunkSize = 500,
        Overlap = 50,
        EmbeddingProvider = "AzureOpenAI",
        TenantField = x => x.TenantId
    });

await collection.InsertAsync(document);

var results = await collection.SearchAsync(
    "What is the leave approval process?",
    tenantId: currentTenantId);
```

This developer experience can be a narrower, more achievable advantage than attempting to outbuild established general-purpose databases.

12. Strategic options and recommendation

Option	Benefits	Limitations	Assessment
A. New database engine from scratch	Maximum learning, control and research value	Years of correctness, recovery, tooling and operations work before production maturity	Good learning/research project; weak first commercial path
B. PostgreSQL extension plus services	Keeps SQL/ACID maturity while enabling custom types, functions and workers	Extension development and deployment complexity	Strong technical product route
C. AI data platform on PostgreSQL/pgvector	Fastest MVP; focuses effort on workflow, security and developer value	Underlying engine is not proprietary; dependency on PostgreSQL ecosystem	Recommended first product route
D. SQL Server/Azure-specific layer	Excellent fit for Microsoft enterprises and current skills	Narrower platform and product dependency	Possible later enterprise edition/integration

12.1 Recommended product statement

Recommended positioning

The easiest secure transactional AI data platform for .NET teams: PostgreSQL-compatible storage, automatic AI indexing, permission-aware hybrid retrieval, embedding lifecycle management and complete retrieval observability.

12.2 Why PostgreSQL first

- Avoids rebuilding ACID transactions, recovery, SQL, indexes, backups and replication
- pgvector already supports exact search and HNSW/IVFFlat approximate indexes
- PostgreSQL provides JSON and full-text capabilities in the same engine
- Broad tooling, hosting and language ecosystem
- Allows differentiated effort to go into AI lifecycle, security and developer experience

13. Risks and success criteria

13.1 Major risks

- Feature convergence: established database vendors continue adding AI pipeline features.
- Search quality: vector similarity alone may produce poor results without chunking, lexical retrieval and reranking.
- Security: filtering after retrieval can leak unauthorized context or metadata.
- Model dependency: providers, dimensions and pricing change over time.
- Operational cost: embeddings, indexes and reranking can be expensive.
- Scope growth: attempting to support all SQL, all models and distributed operation will stall the MVP.
- Reliability: background ingestion must handle retries, idempotency, deletion and partial failure.
- Benchmarking: a product needs measurable retrieval and developer-productivity improvements, not only an attractive architecture.

13.2 MVP success criteria

- A .NET application can create an AI collection and ingest content with minimal code.
- Normal rows, JSON metadata and embeddings remain transactionally connected.
- Queries can combine tenant/security filters, keyword search and vector similarity.
- Source updates reliably create new chunks/embeddings and retire stale ones.
- A retrieval inspector explains selected results and scores.
- The system survives worker restarts without duplicate or missing chunks.
- A demo RAG application demonstrates permission-aware retrieval end to end.
- Performance and retrieval quality are compared against direct pgvector usage to prove added value.

13.3 Test strategy

- Unit tests for chunking, content hashes, distance calculations and policy evaluation
- Integration tests against PostgreSQL/pgvector
- Crash/restart and idempotency tests for ingestion workers
- Security tests proving cross-tenant content cannot be retrieved
- Golden retrieval datasets for relevance regression
- Load tests for concurrent ingestion and search
- Cost and latency dashboards by model/provider/index

14. Roadmap and next actions

14.1 Product phases

10. Prototype: PostgreSQL/pgvector, .NET API, ingestion, exact/hybrid search and one embedding provider.
11. MVP: secure collections, lifecycle management, retries, HNSW, SDK and retrieval inspector.
12. Pilot: enterprise authentication, audit, backup guidance, evaluation datasets and operational dashboards.

13. Expansion: rerankers, more providers, richer parsers, GraphRAG/relationships and optional SQL Server integration.
14. Advanced: extension-level optimizations, distributed workers and carefully selected in-database AI functions.

14.2 Immediate engineering backlog

- Create the solution and Docker Compose environment with PostgreSQL and pgvector.
- Define document, chunk, embedding job, index version and retrieval audit schemas.
- Implement tenant-safe repository and query abstractions.
- Implement deterministic chunking and content hashing.
- Add an embedding-provider interface and one provider implementation.
- Implement vector and full-text retrieval with score fusion.
- Build a .NET SDK with AICollection and SearchAsync APIs.
- Build a React page for ingestion status and retrieval inspection.
- Create a small evaluation corpus and measurable relevance tests.
- Document the boundary between the platform and the underlying PostgreSQL database.

14.3 Decision checkpoint

Go/no-go question

After the prototype, continue only if the platform clearly reduces development effort or improves security, observability or retrieval quality compared with using PostgreSQL and pgvector directly.

15. References

Official and primary documentation consulted for the market findings (accessed June 2026):

15. **pgvector project documentation** - Exact and approximate nearest-neighbour search, HNSW, IVFFlat, vector types and PostgreSQL integration.
<https://github.com/pgvector/pgvector>
16. **PostgreSQL pgvector 0.8.2 release** - PostgreSQL-hosted confirmation of the current pgvector release and purpose.
<https://www.postgresql.org/about/news/pgvector-082-released-3245/>
17. **Microsoft SQL Server vector search and vector index** - Native vector type and current availability/preview details for approximate indexing.
<https://learn.microsoft.com/en-us/sql/sql-server/ai/vectors?view=sql-server-ver17>
18. **Microsoft EF Core SQL Server vector search** - SQL Server 2025 vector support from the .NET/EF Core perspective.
<https://learn.microsoft.com/en-us/ef/core/providers/sql-server/vector-search>
19. **Oracle AI Vector Search overview** - Combining semantic search over unstructured content with relational business data.
<https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/overview-node.html>
20. **MongoDB Vector Search overview** - ANN/ENN, full-text combination and field filtering.
<https://www.mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/>

21. **SingleStore vector functions** - Native vector type, kNN and ANN search.
<https://docs.singlestore.com/db/v9.0/reference/sql-reference/vector-functions/using-vector-functions/>
22. **SingleStore hybrid search** - Full-text and vector blending/reranking.
<https://docs.singlestore.com/cloud/developer-resources/functional-extensions/hybrid-search-re-ranking-and-blending-searches/>
23. **Elasticsearch hybrid search** - Full-text plus vector retrieval and reciprocal rank fusion.
<https://www.elastic.co/docs/solutions/search/hybrid-search>
24. **Elasticsearch vector search** - Vector database functionality combined with full-text, filters and aggregations.
<https://www.elastic.co/docs/solutions/search/vector>
25. **MySQL HeatWave Vector Store** - In-database vector store and RAG workflows.
<https://dev.mysql.com/doc/heatwave/en/mys-hw-genai-vector-store-overview.html>
26. **MySQL HeatWave vector index creation** - Automatic vector index creation, including HNSW in current documentation.
<https://dev.mysql.com/doc/heatwave/en/mys-hw-genai-vector-index-creation.html>

Appendix A - Illustrative API and SQL

```
await using var connection = new HybridDbConnection(
    "Server=localhost;Database=app");

var results = await connection.QueryAsync<DocumentResult>(
    """
    SELECT id, title
    FROM documents
    WHERE tenant_id = @tenantId
    AND category = @category
    ORDER BY embedding <-> @embedding
    LIMIT 5
    """,
    new { tenantId, category = "Azure", embedding = queryEmbedding });

CREATE AI COLLECTION policies (
    id UUID PRIMARY KEY,
    tenant_id UUID,
    title TEXT,
    content TEXT,
    metadata JSON
)
WITH (
    embedding_column = content,
    chunk_size = 500,
    overlap = 50,
    access_filter = tenant_id
);
```

Appendix B - From-scratch milestone checklist

Milestone 1: durable key-value engine

- Put/Get/Delete
- Append-only log
- Checksums
- Startup recovery

- Compaction
- One million record durability test

Milestone 2: table engine

- Catalog and schemas
- Typed binary rows
- Primary keys
- CRUD
- Restart correctness

Milestone 3: minimal SQL

- Tokenizer/parser/AST
- CREATE/INSERT/SELECT/UPDATE/DELETE
- WHERE/ORDER BY/LIMIT
- Table scans

Milestone 4: vectors

- VECTOR type
- Dimension validation
- Cosine/L2/dot product
- Exact top-K
- Structured filtering

Milestone 5: transactions

- BEGIN/COMMIT/ROLLBACK
- WAL
- Recovery
- Basic locks

Milestone 6: indexes and productization

- Hash and B+ tree indexes
- HNSW/IVF
- Query planner
- Server, SDK, CLI, Studio
- Backup and restore